The Spring Cloud project offers a few options for adding client-side service discovery to your Spring applications. One of the popular choices is Spring Cloud Netflix Eureka, which wraps the Eureka service registry developed by Netflix. Alternatives are Spring Cloud Consul, Spring Cloud Zookeeper Discovery, and Spring Cloud Alibaba Nacos.

Besides managing a service registry explicitly, you'll also need to add the correct integration to all your applications. For each of the previously mentioned options, Spring Cloud provides a client library you can add to your Spring application so it can use the service registry with minimal effort. Finally, Spring Cloud Load Balancer can be used for client-side load balancing, which is the preferred choice over Spring Cloud Netflix Ribbon (no longer maintained).

All those libraries offered by Spring Cloud contributed to making it an excellent choice for building cloud native applications and implementing microservices architectures. A benefit of such a solution is that your applications have complete control over the load-balancing strategy. Suppose you need to implement patterns like *hedging*: sending the same request to multiple instances to increase the chance one responds correctly within a specific time limit. Client service discovery can help you with that.

A drawback is that client service discovery assigns more responsibility to developers. If your system includes applications built using different languages and frameworks, you'll need to handle the client part of each of them in different ways. Also, it results in one more service to deploy and maintain (the service registry), unless you use PaaS solutions like Azure Spring Apps or VMware Tanzu Application Service, which provide it for you. Server-side discovery solutions solve these issues at the expense of fine-grained control in the application. Let's see how.